

Sharing intermediate sums in the finite integer-multiplication network

Douglas Colkitt*

October 7, 2026

Abstract

A cancellation-free computation graph replaces the separate rectangle mixers in our preceding finite bit network. Reusing roles along computation paths needs one auxiliary role per output plus one per binary addition. Source supports and reachable-target supports supply non-degenerate frames for the forward and reversed stages, without extra rank loss. The resulting conditional integer-multiplication saving is $\kappa = 2^{-63}$, a factor 16 above the prior 2^{-67} witness. No practical runtime improvement or optimality is claimed.

1 Scope and construction

We retain the fixed finite-alphabet, fixed finite number of one-dimensional tapes model and assume the unchanged interfaces of OpenAI’s *Integer multiplication below $n \log n$* , pinned at

adc7f1241b42e322a6451854ab7e4b4c146bf78a.

The complex network, numerical maps, and final rounding are unchanged.

1.1 Shared intermediate sums for the bit network

We retain $h = 46$, $v = \binom{h}{3} = 15180$, $N = v^3$, $m = h^3 = 97336$, and the rational form $I - J/9$ on $F = \mathbb{Q}^h$. Only the bit network changes; the complex primitive is retained. Fix a common point i of a neighboring triple pair. On the other $n = h - 1$ points, introduce inputs x_{ab} indexed by unordered pairs and compute all outputs

$$z_{cd} = \sum_{\{a,b\} \cap \{c,d\} = \emptyset} x_{ab} \quad (\{c,d\} \in \binom{[n]}{2}).$$

The sums are over $\mathbb{F}_2$. We use a circuit in which the two input supports of every binary addition are disjoint. Equal formal sums are shared as single nodes. Thus the circuit has no cancellation: a source contributes to an output exactly when a directed path joins them, and that path is unique.

An explicit construction uses a balanced recursive split of the point set. For a vector of disjoint-support values, compute the total and all sums omitting one or two positions. The leave-two-out routine splits the vector in half: exclusions in one half use that half’s corresponding sum plus the other half’s total; split exclusions combine the two leave-one-out sums. Totals use balanced binary addition. When only leave-one-out sums are needed, use prefix and suffix sums, adding the prefix before a position to the suffix after it. Zero terms are omitted throughout.

*Prepared with assistance from OpenAI Codex. This is not independent review or endorsement by OpenAI.

For pair inputs, return sums excluding zero, one, or two vertices. Recursively compute these quantities for the left-left and right-right input pairs. The cross pairs form a matrix. Its row sums and column sums feed the vector routine just described. To omit one row and one column, apply leave-one-out first within each row and then down each column of the resulting matrix. For two excluded vertices in the left half, combine the recursive left-left answer, the right-right total, and the cross-matrix row sum omitting those two rows. The right-half case is symmetric. For one excluded vertex in each half, combine the two recursive leave-one-out answers with the cross-matrix answer omitting that row and column. Single exclusions and totals combine the same three disjoint input regions. This proves correctness by induction.

Use the natural point order and splits at $\lfloor n/2 \rfloor$, and identify nodes whenever their formal input-support sets agree. Discard nodes not feeding a requested leave-two-out output. The supplied implementation records every support as an integer bit set and checks disjointness at every addition. At $n = 45$ the resulting circuit has

$$p = 990 \text{ inputs, } q = 990 \text{ outputs, } c = 11566 \text{ active binary additions.}$$

Exact support comparison checks all 990 outputs, including all 893970 nonzero matrix coefficients and all required zeros. The construction is a finite explicit circuit; these counts make no optimality claim.

1.2 Reversible embedding and the auxiliary-role count

We next give a reusable transfer for such circuits. Treat each use of a node by a later addition, or by a designated output, as an outgoing edge. Remove unused nodes. At an input node, allocate one role per outgoing edge, select one pivot as its input slot, and copy the pivot into the other outgoing roles. At a binary addition node, the two incoming roles already exist. Reuse the first as the pivot for one outgoing edge and allocate a fresh role for every other outgoing edge. Add the second input into the pivot and then copy the pivot into the other outgoing roles. The second incoming role is thereafter inactive during the forward circuit. Each node operation is an invertible linear map, with inverse obtained by undoing fanout and then the addition.

This rule identifies roles only along directed paths. In particular, two simultaneous inputs never coincide. If there are c addition nodes and q designated outputs, the total number of outgoing edges is $2c + q$. One role identification is made at each addition node, giving

$$R_{\text{local}} = c + q = 12556.$$

Equivalently, an input contributes its outdegree in new roles and an addition contributes its outdegree minus one. Inputs with one outgoing edge may be regarded as identity gates, so every role participates in the finite mixer. Let L be the product of these node maps in topological order, V the copy into input slots, and J the injection from designated output slots. Then JLV is precisely the disjoint-pair map. Each of the h choices of common point gets its own circuit, so

$$R_{\text{side}} = h(c + q) = 577576$$

side roles suffice per invocation.

Arbitrary initial scratch is handled by the same transparent schedule as for the rectangle circuit. With G, R denoting the old central gather and scatter, the chronological operations are

$$L, J, L^{-1}, R, V, G, R, L, J, L^{-1}, G, V.$$

An initial side vector z contributes JLz in the early injection and $JL(z + Vx)$ in the later one, leaving exactly $JLVx$. The final inverse mixer and copy restore z ; centers are restored as before. The complete side map is the intersection-one neighbor matrix, since each neighbor has a unique common point. Thus $JLV + RG = I$ and the invocation is $y \leftarrow y + x$ on all inputs.

1.3 Frames from source and target supports

Write the original tensor decomposition at a stage as $A = B \perp P$, with P and the future factor Q nondegenerate lines. Suppress Q and put

$$D_0 = B \otimes F, \quad D_1 = A \otimes F, \quad D_U = (B \otimes F) \perp (P \otimes U).$$

During the middle forward application of L , assign each node the span U of the physical source triple indicators reaching it. At an input node this is its source line. Along every dataflow edge the source-support sets grow, so these spans are nested. Along the physical roles, which follow paths through the circuit, the labels are therefore increasing. A role allocated at a branching node enters from D_0 ; a retired input role subsequently grows to D_1 in the final cleanup.

Every triple in a source-support span contains its circuit's common point i . For any vector u in such a span, $\sum_j u_j = 3u_i$, and hence

$$\langle u, u \rangle = \sum_{j \neq i} u_j^2 > 0 \quad (u \neq 0).$$

Thus all these spans are nondegenerate. At an output, every contributing source triple is a neighbor of the target triple T , giving $U \subset t_T^\perp$. The output role can consequently grow to the original physical target frame without decreasing rank. Every node gate has one common frame on all its incoming and outgoing roles, as required by the common-frame identity.

The reversed stage requires a different span assignment. Reverse and invert the scalar schedule with logical source Y and target X . In the middle L^{-1} , label a node by the span of the physical X target indicators reachable from it in the forward dataflow graph. These descendant sets grow along reversed edges. Every such triple still contains the same point i , so its span is nondegenerate by the same norm identity. At an original input, all its reachable targets are neighbors of its physical Y triple; the span therefore lies in that triple's orthogonal complement. This verifies the reverse middle computation as well as its endpoints. The source-support frames of the forward computation are not incorrectly reused in reverse.

All early mixers and their inverses use $D_0 \otimes Q$, and all late cleanup mixers and inverses use $D_1 \otimes Q$. The data-copy and injection gates are grouped by data triple, with the same individual line and orthogonal-complement frames as in the rectangle construction. Physical X grows from $A \otimes \langle t \rangle \otimes Q$ to $(A \otimes F) \otimes Q$; physical Y grows from $B \otimes \langle t \rangle \otimes Q$ to $((B \otimes F) \perp (P \otimes t^\perp)) \otimes Q$. These are exactly the original stage boundaries. The only decreasing edges remain the h central returns per invocation, each losing dimension h . Consequently $L_{\text{dec}} = 3v^2h^2$ is unchanged.

1.4 Stage sharing and rank accounting

Use a bijection π of triples with $|T \cap \pi(T)| = 1$. One explicit choice pairs the ground points consecutively. For a triple with a full pair and a singleton, keep the singleton and cycle the full pair among all pairs other than the singleton's pair. Otherwise keep the point in the least indexed pair and flip the other two to their partners. This is a bijection on the two disjoint classes, with the stated intersection property.

Share the entire auxiliary bank of the stage-1 invocation with fixed triples (A, B) with that of stage 3 at $(B, \pi(A))$, keeping each local role index. Every invocation restores arbitrary scratch, and the sharing is bijective. The joining labels are

$$E = F \otimes \langle t_A \rangle \otimes \langle t_B \rangle, \quad H = \langle t_B \otimes t_{\pi(A)} \rangle^\perp \otimes F.$$

They are nondegenerate and $E \subset H$ since $t_A \perp t_{\pi(A)}$. Replacing the two former terminal edges by this join saves exactly m in edge rank per removed role. Thus

$$\begin{aligned} W_b^{\text{dag}} &= 2N + 2v^2(R_{\text{side}} + h) = 273201575169600, \\ s_b^{\text{dag}} &= W_b^{\text{dag}}m - N + 6v^2h^2 = 26592347948314104000, \\ W_b^{\text{dag}}m - s_b^{\text{dag}} &= 572394081600 > 0. \end{aligned}$$

Proposition 1 (Shared-computation bit interface). *The new bit network exchanges the two scalar data banks, restores every auxiliary value, and satisfies the original all-role endpoint identity with W_b^{dag} roles and total edge rank s_b^{dag} .*

Proof. The transparent schedule proves the scalar shear and arbitrary restoration. Forward, inverse, forward invocations give the bank exchange. The support labels and their reverse counterparts establish all common frames and nested side transitions. Central decreases are unchanged. Telescoping therefore gives $W_b^{\text{dag}}m - 2N + 2L_{\text{dec}}$ absolute label change; the original rational source correction adds N . Every surviving auxiliary role still has endpoint matrices zero and I_m and is fixed by the scalar permutation; the data endpoints are unchanged. The original common-frame and rational-shear transfer consequently applies. All compiled gates and bases are fixed finite data, independent of the eventual input length. $\square$

1.5 Explicit rational bounds for the two recurrences

Put

$$a_b = \frac{187}{10^{11}}, \quad a_c = \frac{9}{500000000000}, \quad L_0 = \frac{5743}{500}.$$

The exact deficits satisfy

$$\eta_b^{\text{dag}} = \frac{27}{1254369032} > a_b L_0, \quad \eta_c = \frac{7}{22253827054} > a_c L_0.$$

The retained rational logarithm enclosure proves $\log m < L_0$. Therefore $m^{1-a} > m(1 - aL_0) > m(1 - \eta)$ for either corresponding pair, and we may take

$$\tau = 1 - a_b, \quad \sigma = 1 - a_c. \tag{1}$$

This gives $s_b^{\text{dag}}/W_b^{\text{dag}} < m^\tau$ and the unchanged complex bound $s_c/W_c < m^\sigma$.

2 Conditional exponent and checks

The logarithm enclosure used above follows from

$$2 \sum_{j=0}^{23} \frac{z^{2j+1}}{2j+1} \leq \log x \leq 2 \sum_{j=0}^{23} \frac{z^{2j+1}}{2j+1} + \frac{2z^{49}}{49(1-z^2)}, \quad z = \frac{x-1}{x+1}, \quad 1 \leq x \leq 2,$$

applied to 2 and $m/2^{16}$ with exact rational arithmetic. With $a = a_b$, take

$$\epsilon = 1/21, \quad \beta = 9/10, \quad c = 9a/10, \quad \lambda = 1 - 19a^2/20, \quad \lambda' = 1 - 9a^2/10, \\ \delta = 1/16, \quad C_1 = 20.$$

Then $\tau(1+c/\beta) = 1 - a^2 < \lambda < \lambda' < 1$; the separate complex conditions follow from $19a^2/20 < a_c$ and $9a^2 < a_c$. The exact minimum of the seven exponent margins is

[illegible]

The remaining margins are $g_1 > 1/2$, $g_4 = 20a/21$, $g_5 = 13/112$, $g_6 = 299/336$, and $g_7 = 1/21$. The strict gap absorbs fixed logarithmic factors, giving $O(n(\log n)^{1-2^{-63}})$ under the retained assumptions. The previous dimension, stopping, and guard audits still apply: the guard is $p^{20/21} = o(p)$ and $K = \Theta(p^{3a/70}) = o(\ell)$ for $\ell = \Theta(p^{20/21})$, with $K/\log p \rightarrow \infty$.

The companion artifacts are

- `scripts/exclusion_circuit.py` and `scripts/dag_network.py`;
- `certificates/dag-network.json`;
- `patches/h46-dag-63.patch`.

The certificate checks every output support at $n = 45$, the reversible slot assignment, and both directions of support nesting. Small complete invocations are checked on every input basis vector, including dirty scratch. A full small three-stage model checks auxiliary sharing and bank exchange. These checks support the written transfer argument and do not formally verify the complete upstream multiplication theorem.